Mini Cloud Storage Backend

A simple backend system that simulates a cloud file storage service with upload, delete, file listing, storage summary and deduplication logic.

This project implements file metadata storage with a **500 MB user quota**, deduplication using file hashes, and concurrency-safe operations.

🚀 Project Setup

📥 1. Clone Repository

```
git clone https://github.com/SouravGolder/mini_cloud_storage_backend.git
cd mini_cloud_storage_backend
```

📦 2. Install Dependencies

```
npm install
```

🔐 3. Environment Variables

Create a .env in project root:

```
PORT=3000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=mini_cloud
DB_PORT=3306
```

🛠️ 4. Database Setup

Create Database:

```
CREATE DATABASE mini_cloud;
USE mini_cloud;
```

Create Tables:

```sql
-- Users Table
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

-- Files Table (physical storage)
CREATE TABLE files (
    id SERIAL PRIMARY KEY,
    file_size BIGINT NOT NULL,
    file_hash VARCHAR(64) NOT NULL UNIQUE
);

-- UserFiles Table (ownership)
CREATE TABLE user_files (
    id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(id) ON DELETE CASCADE,
    file_id INT REFERENCES files(id) ON DELETE CASCADE,
    file_name VARCHAR(255) NOT NULL,
    upload_time TIMESTAMP DEFAULT NOW(),
    UNIQUE(user_id, file_name)
);
```

🚀 5. How to Run

```bash
node index.js
```

📡 API Endpoints Test (Postman)

➕ 1. Upload File

URL: **POST** http://localhost:3000/users/:user_id/files

Body (JSON)

```json
{
  "file_name": "example.txt",
  "file_size": 12345,
  "file_hash": "abcdef123456"
}
```

Rules

- File name must be unique per user
- Cannot exceed 500 MB quota
- Deduplicates by file hash
- Same user + same hash → updates file name

🗑 2. Delete File

URL: **DELETE** /users/:user_id/files/:file_id

- Deletes user-file reference
- Deletes physical file only if no other users reference it

📊 3. Storage Summary

URL: **GET** /users/:user_id/storage-summary

📄 4. List User Files

URL: **GET** /users/:user_id/files

□ Design Decisions

📌 1. Deduplication

Files are identified by a file hash. If two users upload files with the same hash:

- Only one record is stored in the files table
- Multiple references are stored in user_files

This reduces storage redundancy.

📌 2. User Quota

- Each user has a 500 MB quota.
- Before upload, total storage is calculated
- Upload is rejected if it exceeds limit

📌 3. Replace Same File

If the same user uploads a file with the same hash:

- Existing record is updated like file name. But the size of existing and uploaded file must be same since the hash is same for both file.
- No duplicate entry is created

⚙ Concurrency Control

To handle simultaneous uploads:

- Database transactions are used
- Row-level locking (FOR UPDATE) ensures safe updates
- file_hash has a UNIQUE constraint

Result:

- Prevents duplicate files
- Prevents storage overflow during concurrent uploads

🚀 Scaling to 100K Users

When the number of users grows to **100K or more**, the system can experience high load and slower performance. To ensure scalability and efficiency, the following strategies can be applied:

📌 1. Database Optimization

Improve database performance by adding **indexes** on frequently queried fields:

- `user_id`
- `file_id`
- `file_hash`
- Using **read replicas** to handle heavy read operations and reduce pressure on the main database

□ 2. Caching

Use **Redis** to cache frequently accessed data such as:

- Storage summary
- File lists

Benefits:

- Reduces database queries
- Improves response time
- Enhances overall performance

🎁 3. Horizontal Scaling using Multiple Servers

Handle increased traffic by **Running multiple backend server instances**

- Using a load balancer (e.g., **Nginx**) to distribute requests evenly across servers

💼 4. Storage Optimization

- Store actual file content in cloud storage (e.g., **AWS S3**)
- Keep only file metadata in the database

Benefits:

- Improves scalability
- Reduces database storage load
- Ensures better file management